

Zaawansowane Technologie Baz Danych

Porównanie wydajności systemów zarządzania bazami danych:
PostgreSQL, MySQL, MongoDB, Redis

Sprawozdanie projektowe

Politechnika Krakowska
Kierunek: Informatyka

Kwiecień 2026

Spis treści

1	Cel i zakres pracy	3
1.1	Hipoteza badawcza	3
2	Opis wybranych systemów zarządzania bazami danych	3
2.1	PostgreSQL 16	3
2.2	MySQL 8.0	3
2.3	MongoDB 7	4
2.4	Redis 7	4
3	Awaryjność, bezpieczeństwo, migracje, integracje i skalowalność	5
4	Opis zbioru danych	5
4.1	Modele danych w systemach nierelacyjnych	6
4.2	Skale danych testowych	7
5	Opis aplikacji testowej	7
5.1	Wymagania	7
5.2	Wykorzystane technologie i narzędzia	7
5.3	Struktura projektu i opis działania	7
6	Opis scenariuszy testowych	8
7	Wyniki testów wydajnościowych	9
7.1	Scenariusze CREATE – porównanie per scenariusz	9
7.2	Scenariusze READ – porównanie per scenariusz	10
7.3	Scenariusze UPDATE – porównanie per scenariusz	11
7.4	Scenariusze DELETE – porównanie per scenariusz	12
7.5	Porównanie: wpływ indeksów na wydajność (skala 1M)	12
8	Wizualizacja wyników	13
9	Analiza indeksów	20
9.1	Zastosowane indeksy	20
9.2	Analiza planów zapytań (EXPLAIN)	20
10	Weryfikacja hipotezy badawczej	21
10.1	Analiza wyników – PostgreSQL (skala 1M)	21
10.2	Wnioski dotyczące hipotezy	21
11	Rozszerzona analiza wyników	22
11.1	Skalowalność	22
11.2	Porównanie modeli danych	22
11.3	Dane półustrukturalne (JSON)	23
11.4	Automatyzacja testów	23
12	Podsumowanie i wnioski	23

1 Cel i zakres pracy

Celem projektu jest porównanie wydajności czterech systemów zarządzania bazami danych (SZBD) przy wykonywaniu operacji CRUD (Create, Read, Update, Delete) na dużych zbiorach danych. Analizie poddano:

- **2 systemy relacyjne:** PostgreSQL 16, MySQL 8.0
- **2 systemy nierelacyjne:** MongoDB 7 (dokumentowa), Redis 7 (klucz-wartość)

Zakres pracy obejmuje:

1. Zaprojektowanie schematu danych systemu szpitalnego (13 tabel relacyjnych, model dokumentowy MongoDB, struktury klucz-wartość Redis).
2. Implementację aplikacji testowej automatyzującej generowanie danych, wstawianie, benchmarki i analizę.
3. Przeprowadzenie 24 scenariuszy testowych CRUD (6 na każdą operację) w trzech skalach: 10 000, 100 000, 1 000 000 rekordów.
4. Porównanie wydajności przed i po zastosowaniu indeksów.
5. Analizę planów zapytań (EXPLAIN).
6. Weryfikację hipotezy badawczej o wpływie indeksów na wydajność.

1.1 Hipoteza badawcza

H1: Indeksy a wydajność – Zastosowanie indeksów znacząco poprawia wydajność operacji SELECT kosztem spadku wydajności operacji INSERT i UPDATE.

2 Opis wybranych systemów zarządzania bazami danych

2.1 PostgreSQL 16

PostgreSQL jest zaawansowanym, obiektowo-relacyjnym systemem baz danych o otwartym kodzie źródłowym. Oferuje pełną zgodność z ACID, zaawansowane mechanizmy indeksowania (B-tree, Hash, GIN, GiST, BRIN), obsługę JSON/JSONB, widoki zmateriaлизованne, partycjonowanie tabel oraz replikację logiczną i strumieniową.

Zalety: Zaawansowane typy danych, rozbudowany optymalizator zapytań, pełna zgodność ze standardem SQL, doskonała obsługa transakcji, MVCC (Multi-Version Concurrency Control), wsparcie dla procedur składowanych w wielu językach (PL/pgSQL, PL/Python).

Wady: Wyższe zużycie pamięci w porównaniu z MySQL, wolniejsze wstawianie danych masowych bez odpowiedniej konfiguracji, bardziej złożona konfiguracja.

Zastosowania biznesowe: Systemy finansowe, systemy ERP, analityka danych, systemy GIS (PostGIS), platformy e-commerce.

2.2 MySQL 8.0

MySQL jest najpopularniejszym relacyjnym systemem baz danych o otwartym kodzie. Wykorzystuje silnik InnoDB z obsługą transakcji ACID, mechanizmem MVCC, kluczami

obcymi i indeksami B-tree. MySQL 8.0 wprowadził okna analityczne, CTE (Common Table Expressions) i ulepszony optymalizator.

Zalety: Wysoka wydajność operacji odczytu, łatwość konfiguracji, szeroka kompatybilność, duża społeczność, niskie wymagania sprzętowe.

Wady: Ograniczone wsparcie dla zaawansowanych typów danych, brak niektórych funkcji SQL standardu (np. pełna obsługa sekwencji), historycznie słabsza optymalizacja podzapytań.

Zastosowania biznesowe: Aplikacje webowe (WordPress, Joomla), systemy CMS, platformy SaaS, systemy logistyczne.

2.3 MongoDB 7

MongoDB jest dokumentową bazą danych NoSQL, przechowującą dane w formacie BSON (Binary JSON). Oferuje elastyczny schemat, replikację zestawów replik (Replica Sets), sharding dla skalowalności horyzontalnej oraz zaawansowane indeksowanie (pojedyncze, złożone, wielokluczowe, tekstowe, geoprzestrzenne).

Zalety: Elastyczny schemat (schema-less), naturalne modelowanie danych hierarchicznych, szybkie operacje na pojedynczych dokumentach, łatwa skalowalność horyzontalna, wbudowana agregacja.

Wady: Brak transakcji wielodokumentowych (do wersji 4.0), większe zużycie pamięci dyskowej, brak JOIN (wymaga denormalizacji lub \$lookup), ewentualna spójność w konfiguracjach rozproszonych.

Zastosowania biznesowe: Systemy zarządzania treścią, aplikacje mobilne, IoT, katalogi produktów, systemy analityki czasu rzeczywistego.

2.4 Redis 7

Redis (Remote Dictionary Server) jest bazą danych typu klucz-wartość działającą w pamięci RAM. Obsługuje struktury danych: stringi, hashe, listy, zbiory, zbiory uporządkowane, strumienie. Oferuje persystencję (RDB, AOF), replikację master-slave i klaster Redis.

Zalety: Ekstremalnie niska latencja (< 1 ms), bogate struktury danych, atomowość operacji, Pub/Sub, obsługa TTL (Time-To-Live), skrypty Lua.

Wady: Ograniczona pojemność (limitowana pamięcią RAM), brak złożonych zapytań (brak SQL), brak relacji między danymi, utrata danych w razie awarii (zależy od konfiguracji persystencji).

Zastosowania biznesowe: Cache'owanie, sesje użytkowników, kolejki wiadomości, tablice wyników w grach, systemy czasu rzeczywistego, liczniki i metryki.

3 Awaryjność, bezpieczeństwo, migracje, integracje i skalowalność

Tabela 1: Porównanie cech niefunkcjonalnych analizowanych SZBD

Cecha	PostgreSQL	MySQL	MongoDB	Redis
Awaryjność	WAL, Point-in-Time Recovery, replikacja strumieniowa	InnoDB Crash Recovery, replikacja master-slave, Group Replication	Replica Sets z auto-failover, journal	RDB snapshots, AOF, Sentinel dla HA
Bezpieczeństwo	Role, uprawnienia, SSL/TLS, Row-Level Security, szyfrowanie at-rest	Role, uprawnienia, SSL/TLS, szyfrowanie InnoDB tablespace	SCRAM-SHA-256, Role-Based Access, szyfrowanie at-rest, SSL/TLS	ACL (od v6), hasła, SSL/TLS, brak szyfrowania at-rest
Migracje	pg_dump/pg_restore, replikacja logiczna, pgLoader	mysqldump, MySQL Shell, replikacja	mongodump/mongoimport, Atlas Live Migration	RDB/AOF, redis-cli -pipe, Redis Data Integration
Integracje	Foreign Data Wrappers, Kafka, Debezium, REST API	Connectors (JDBC, ODBC), MySQL Router	Connectors natywne, Atlas Triggers, Kafka Connector	Pub/Sub, Streams, integracja z Celery, Sidekiq
Skalowalność	Skalowanie pionowe, Citus dla horyzontalnego, partycjonowanie	Skalowanie pionowe, MySQL NDB Cluster, ProxySQL	Sharding natywny, skalowalność horyzontalna	Redis Cluster (do 1000 węzłów), sharding automatyczny

4 Opis zbioru danych

Dane testowe modelują system informatyczny szpitala. Schemat relacyjny składa się z **13 tabel**:

Tabela 2: Schemat relacyjny – tabele systemu szpitalnego

Tabela	Typ	Opis / klucze obce
departments	Słownikowa	Oddziały szpitalne (id, name, phone)
specializations	Słownikowa	Specjalizacje lekarskie (id, name)
diseases	Słownikowa	Choroby z kodem ICD-10 (id, icd10_code, name)
medical_services	Słownikowa	Usługi medyczne z ceną bazową
medications	Słownikowa	Leki (id, name, active_substance)
patients	Główna	Pacjenci (id, PESEL, imię, nazwisko, data ur., płeć)
doctors	Główna	Lekarze → departments, specializations
visits	Transakcyjna	Wizyty → patients, doctors
performed_services	Transakcyjna	Wykonane usługi → visits, medical_services
diagnoses	Transakcyjna	Diagnozy → visits, diseases
prescriptions	Transakcyjna	Recepty → visits
prescription_items	Transakcyjna	Pozycje recept → prescriptions, medications
test_results	Transakcyjna	Wyniki badań → visits

4.1 Modele danych w systemach nierelacyjnych

MongoDB – model dokumentowy z zagnieżdżeniami: Pacjent jest dokumentem głównym, zawierającym tablicę wizyt, a każda wizyta posiada zagnieżdżone tablice: wykonane usługi, diagnozy, recepty (z pozycjami) oraz wyniki badań. Taki model eliminuje konieczność JOINów, ale komplikuje operacje UPDATE na zagnieżdżonych strukturach.

Redis – model klucz-wartość z wieloma typami struktur:

- `patient:{id}` – HASH z danymi pacjenta,
- `patient:visits:{pid}` – SET identyfikatorów wizyt,
- `visit:status:{id}`, `visit:doctor:{vid}` – STRING,
- `session:doctor:{id}` – HASH z danymi lekarza,
- `visit:diag:{vid}` – LIST diagnoz,
- `prescription:{id}`, `service:total:{vid}`, `department:{id}` – HASH.

4.2 Skale danych testowych

Tabela 3: Liczba rekordów w zależności od skali

Tabela	10 000	100 000	1 000 000
departments	10	20	30
specializations	15	25	40
diseases	100	200	500
medical_services	50	100	200
medications	80	150	400
patients	2 000	10 000	80 000
doctors	50	100	500
visits	10 000	100 000	1 000 000
performed_services	~10 000	~100 000	~1 000 000
diagnoses	~10 000	~100 000	~1 000 000
prescriptions	~4 000	~40 000	~400 000
prescription_items	~8 000	~80 000	~800 000
test_results	~6 000	~60 000	~600 000

5 Opis aplikacji testowej

5.1 Wymagania

- Automatyczne generowanie syntetycznych danych medycznych (własny generator).
- Masowe wstawianie danych (Bulk Insert) do 4 systemów bazodanowych.
- Wykonanie 24 scenariuszy CRUD w trybie bez indeksów i z indeksami.
- Pomiar czasu operacji (średnia z 3 prób) z dokładnością do mikrosekundy.
- Automatyczne generowanie raportów EXPLAIN i wykresów porównawczych.

5.2 Wykorzystane technologie i narzędzia

- **Python 3.13** – język implementacji aplikacji.
- **Docker + Docker Compose** – konteneryzacja wszystkich 4 baz danych.
- **psycopg2** – sterownik PostgreSQL.
- **mysql-connector-python** – sterownik MySQL.
- **pymongo** – sterownik MongoDB.
- **redis-py** – sterownik Redis.
- **pandas + matplotlib** – analiza wyników i wizualizacja.
- **L^AT_EX** – skład sprawozdania.

5.3 Struktura projektu i opis działania

Aplikacja działa w pełni automatycznie. Polecenie `python run_all.py` kolejno:

1. Sprawdza połączenia z bazami danych (ping).
2. Dla każdej skali (10k, 100k, 1M): generuje dane, wstawia do baz, uruchamia benchmark bez/z indeksami, generuje EXPLAIN.
3. Łączy wyniki CSV i generuje wykresy porównawcze (per scenariusz).

6 Opis scenariuszy testowych

Zdefiniowano **24 scenariusze testowe** – po 6 dla każdej operacji CRUD. Każdy scenariusz wykonywany jest 3 razy, a raportowana jest średnia arytmetyczna czasu wykonania. Scenariusze zaprojektowano tak, aby były logicznie porównywalne między technologiami.

Tabela 4: Scenariusze testowe – operacje CREATE

Nr	Nazwa	SQL (PG/MySQL)	MongoDB	Redis
C1	insert_patient	INSERT INTO patients	insert_one	pipeline HSET+SADD
C2	insert_visit	INSERT INTO visits (FK)	\$push wizyta	pipeline SET+SET+SADD
C3	insert_diagnosis	INSERT INTO diagnoses	\$push diagnoza	RPUSH na liście
C4	insert_prescription	INSERT recept + pozycji	\$push recepta	pipeline HSET
C5	insert_service	INSERT services	\$push usługa	pipeline HSET+HINCRBYFLOAT
C6	insert_test_result	INSERT test_results	\$push wynik	HSET mapping

Tabela 5: Scenariusze testowe – operacje READ

Nr	Nazwa	SQL (PG/MySQL)	MongoDB	Redis
R1	select_patient_by_id	SELECT by PK	find_one by _id	HGETALL patient:{id}
R2	select_visits_with_doctor	JOIN visits+doctors	find + projection	SMEMBERS + pipeline GET/HGETALL
R3	select_visit_diagnoses	JOIN diagnoses+diseases	aggregate \$unwind	LRANGE + pipeline EXISTS
R4	select_full_history	Multi-JOIN (3 tabele)	find_one + proj.	2-fazowy pipeline (HGETALL+SMEMBERS → GET+LRANGE)
R5	select_aggregated_costs	SUM + GROUP BY	aggregate pipeline	SMEMBERS + batch HGETALL + SUM
R6	select_prescriptions	Triple JOIN	find_one + proj.	SMEMBERS + pipeline GET+HGETALL

Tabela 6: Scenariusze testowe – operacje UPDATE i DELETE

Nr	Nazwa	Opis
U1	update_patient_name	Zmiana nazwiska pacjenta po id
U2	update_visit_status	Zmiana statusu wizyty po id
U3	update_service_price	Zmiana ceny usługi po visit_id
U4	update_diagnosis_notes	Aktualizacja notatki diagnozy po visit_id
U5	update_doctor_license	Zmiana numeru licencji lekarza
U6	update_department_phone	Zmiana telefonu oddziału
D1	delete_test_result	Usunięcie wyniku badania
D2	delete_diagnosis	Usunięcie diagnozy
D3	delete_patient	Usunięcie pacjenta
D4	delete_performed_service	Usunięcie usługi
D5	delete_prescription	Usunięcie recepty
D6	delete_visit_cascade	Usunięcie wizyty (kaskadowe)

7 Wyniki testów wydajnościowych

Poniższe tabele przedstawiają wyniki **poszczególnych scenariuszy osobno** dla każdej bazy danych. Każda wartość to średnia z 3 prób wyrażona w milisekundach [ms].

7.1 Scenariusze CREATE – porównanie per scenariusz

Tabela 7: Czas scenariuszy CREATE [ms] – BEZ indeksów

Scenariusz	Skala	PostgreSQL	MySQL	MongoDB	Redis
insert_patient	10k	2.81	9.15	1.88	0.98
	100k	5.76	13.44	1.63	0.89
	1M	38.43	7.52	1.65	1.22
insert_visit	10k	3.37	7.53	1.41	0.88
	100k	18.62	7.79	0.75	0.84
	1M	142.54	8.16	0.74	0.90
insert_diagnosis	10k	2.08	9.85	2.82	0.51
	100k	2.03	21.20	2.50	0.45
	1M	2.17	7.13	2.72	0.44
insert_prescription	10k	4.43	17.20	0.77	0.86
	100k	6.50	20.42	0.70	0.85
	1M	39.53	13.63	0.70	0.89
insert_service	10k	2.11	16.94	0.74	1.05
	100k	7.90	22.29	0.70	0.86
	1M	2.22	8.56	0.75	0.88
insert_test_result	10k	1.93	10.27	0.70	0.80
	100k	1.97	13.25	0.73	0.78
	1M	1.99	10.11	0.68	0.82

Obserwacja: PostgreSQL `insert_visit` bez indeksów przy 1M rekordów trwa **142.54 ms** – walidacja kluczy obcych (`patient_id`, `doctor_id`) wymaga sekwencyjnego skanowania tabel bez indeksów. **Redis** konsekwentnie najszybszy w CREATE ($\sim 0.4\text{--}1.2$ ms) dzięki operacjom in-memory na prostych strukturach (HSET + SADD pipeline).

7.2 Scenariusze READ – porównanie per scenariusz

Tabela 8: Czas scenariuszy READ [ms] – BEZ indeksów

Scenariusz	Skala	PostgreSQL	MySQL	MongoDB	Redis
select_patient_by_id	10k	0.43	0.90	0.79	0.48
	100k	0.44	0.90	0.83	0.39
	1M	0.54	0.91	0.80	0.42
select_visits_with_doctor	10k	0.93	1.41	2.28	1.92
	100k	3.77	0.93	2.23	1.56
	1M	23.12	1.15	6.17	1.63
select_visit_diagnoses	10k	0.81	0.89	1.30	1.28
	100k	3.62	0.89	0.85	0.79
	1M	23.36	0.89	0.94	0.65
select_full_history	10k	1.66	0.92	0.90	1.22
	100k	7.89	0.99	0.83	1.24
	1M	44.01	1.03	0.73	1.15
select_aggregated_costs	10k	1.79	1.35	0.90	0.97
	100k	8.27	6.94	0.75	0.94
	1M	41.63	0.94	0.75	1.04
select_prescriptions	10k	0.63	0.87	0.74	1.43
	100k	3.05	0.86	0.75	1.34
	1M	17.15	0.85	0.74	1.50

Tabela 9: Czas scenariuszy READ [ms] – Z indeksami

Scenariusz	Skala	PostgreSQL	MySQL	MongoDB	Redis
select_patient_by_id	10k	0.45	0.98	0.81	0.42
	100k	0.58	0.81	0.80	0.39
	1M	0.46	0.87	0.83	0.39
select_visits_with_doctor	10k	0.55	0.95	1.36	1.44
	100k	0.68	18.94	1.35	1.70
	1M	0.66	1.06	1.44	1.64
select_visit_diagnoses	10k	0.48	0.98	0.79	0.73
	100k	0.65	0.87	0.91	0.54
	1M	0.54	0.87	0.81	0.67
select_full_history	10k	0.64	4.07	0.73	1.55
	100k	0.82	31.59	0.75	1.08
	1M	0.82	1.18	0.74	1.25
select_aggregated_costs	10k	0.53	5.83	0.78	1.05
	100k	0.68	44.58	0.75	0.89
	1M	0.80	1.06	0.74	0.99
select_prescriptions	10k	0.81	0.85	0.75	1.31
	100k	0.55	0.96	0.73	1.36
	1M	0.57	1.13	0.74	1.47

Kluczowa obserwacja: PostgreSQL READ przy 1M rekordów:

- Bez indeksów: `select_full_history` = **44.01 ms** (Multi-JOIN z sekwencyjnym skanowaniem)
- Z indeksami: `select_full_history` = **0.82 ms** (Bitmap Index Scan + Hash Join)
- Przyspieszenie: **98%** – 54-krotne przyspieszenie dzięki indeksom.

Redis wykazuje stabilne czasy READ (~ 0.4 – 1.6 ms) niezależnie od skali, co potwierdza $O(1)$ złożoność dostępu do danych in-memory. Scenariusze wielokluczowe (`select_visits_with_doctor` ~ 1.6 ms) wymagają pipeline’ów `SMEMBERS` + `GET`, ale latencja jest nadal niższa niż PostgreSQL bez indeksów.

7.3 Scenariusze UPDATE – porównanie per scenariusz

Tabela 10: Czas scenariuszy UPDATE [ms] – BEZ indeksów vs Z indeksami (skala 1M)

Scenariusz	PostgreSQL		MySQL		MongoDB		Redis	
	Bez	Z idx	Bez	Z idx	Bez	Z idx	Bez	Z idx
update_patient_name	1.14	1.09	3.81	4.07	0.72	0.72	0.42	0.39
update_visit_status	1.02	1.06	1.80	2.60	0.72	0.73	0.38	0.39
update_service_price	43.45	0.85	3.86	4.21	0.93	0.77	0.38	0.39
update_diagnosis_notes	48.84	0.44	1.83	4.32	0.85	0.71	0.72	0.76
update_doctor_license	1.08	1.03	3.85	3.77	0.82	0.74	0.36	0.41
update_department_phone	1.01	1.01	3.85	3.68	0.69	0.71	0.37	0.45

Obserwacja: PostgreSQL scenariusze U3 i U4 (WHERE po `visit_id`) przy 1M bez indeksów wymagają sekwencyjnego skanowania – odpowiednio **43.45 ms** i **48.84 ms**. Z indeksem na `visit_id`: **0.85 ms** i **0.44 ms**. Scenariusze U1, U5, U6 (WHERE po PK) nie zmieniają się, bo klucz główny ma indeks domyślnie. **Redis** osiąga najniższe czasy UPDATE (~ 0.36 – 0.76 ms) – operacje HSET/SET na pojedynczych kluczach mają stałą złożoność $O(1)$.

7.4 Scenariusze DELETE – porównanie per scenariusz

Tabela 11: Czas scenariuszy DELETE [ms] – BEZ indeksów vs Z indeksami (skala 1M)

Scenariusz	PostgreSQL		MySQL		MongoDB		Redis	
	Bez	Z idx	Bez	Z idx	Bez	Z idx	Bez	Z idx
delete_test_result	2.04	2.06	8.59	7.29	1.41	1.39	0.73	0.79
delete_diagnosis	2.00	2.07	7.39	7.08	1.39	1.44	0.87	0.76
delete_patient	47.19	2.25	7.01	8.47	1.42	1.54	0.80	0.77
delete_service	2.49	2.05	6.46	8.91	1.41	1.37	0.71	0.72
delete_prescription	38.65	2.12	7.75	7.14	1.45	1.39	0.72	0.73
delete_visit_cascade	179.22	4.30	14.44	14.47	1.51	1.55	0.78	0.85

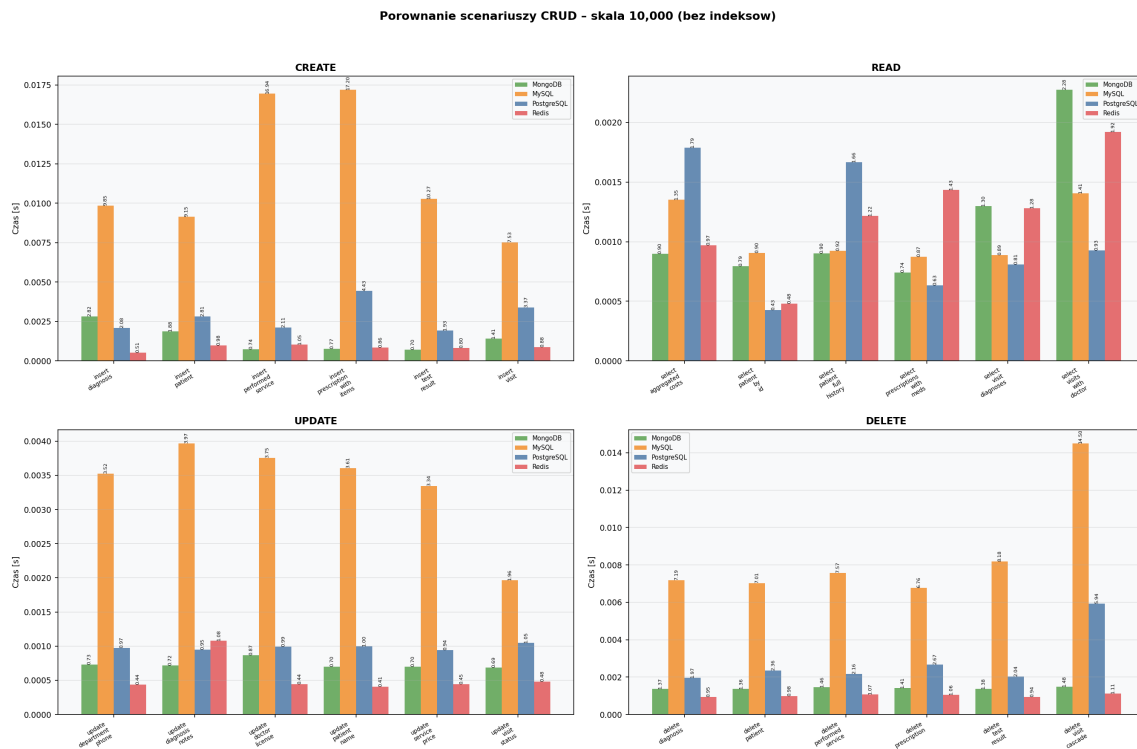
Obserwacja: `delete_visit_cascade` w PostgreSQL bez indeksów = **179.22 ms** (kaskadowe usuwanie wymaga wyszukania powiązanych wierszy w 6 tabelach). Z indeksami: **4.30 ms** – **42-krotne przyspieszenie**. **Redis** `delete_visit_cascade` (~ 0.78 – 0.85 ms) wymaga jawnego usunięcia wielu kluczy pipeline’em, ale operacja jest in-memory i $O(1)$ per klucz.

7.5 Porównanie: wpływ indeksów na wydajność (skala 1M)

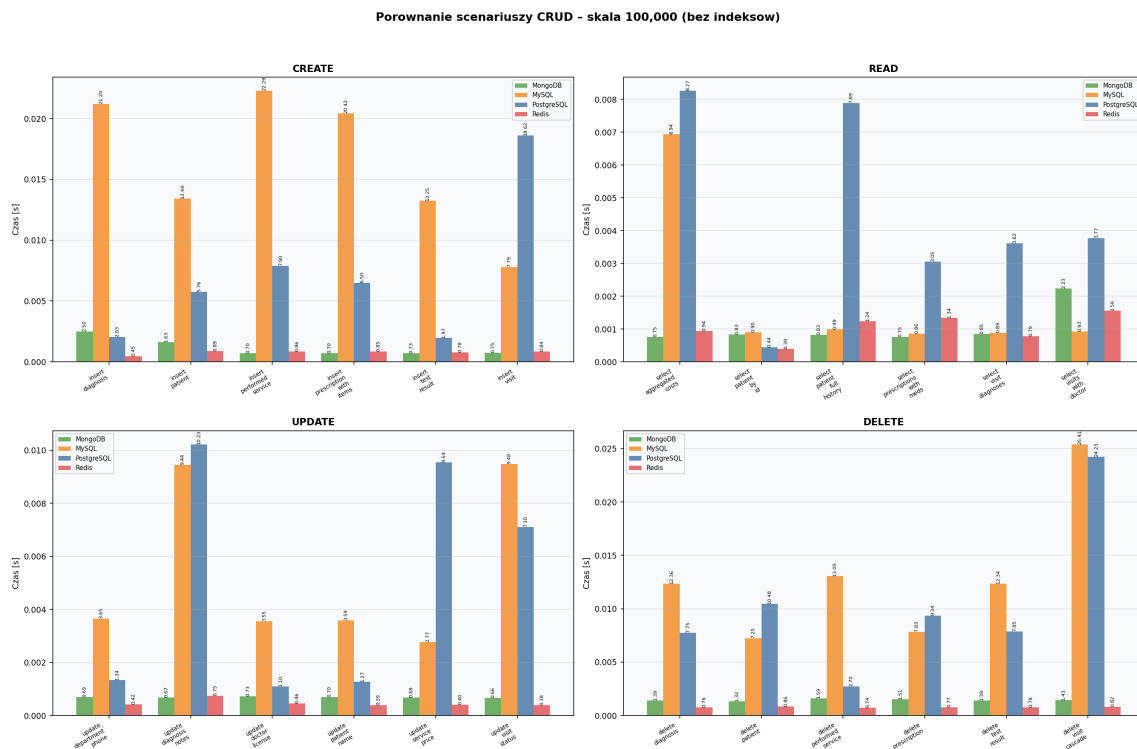
Tabela 12: Zmiana wydajności po zastosowaniu indeksów – wybrane scenariusze (skala 1 000 000)

Scenariusz	PostgreSQL	MySQL	MongoDB	Redis
insert_patient	−93%	+41%	+159%	−34%
insert_visit	−98%	−2%	+21%	−11%
select_visits_with_doctor	−97%	−8%	−77%	0%
select_full_history	−98%	+15%	+1%	+9%
select_aggregated_costs	−98%	+13%	−1%	−4%
update_service_price	−98%	+9%	−17%	+2%
update_diagnosis_notes	−99%	+136%	−16%	+6%
delete_patient	−95%	+21%	+8%	−3%
delete_visit_cascade	−98%	0%	+3%	+9%

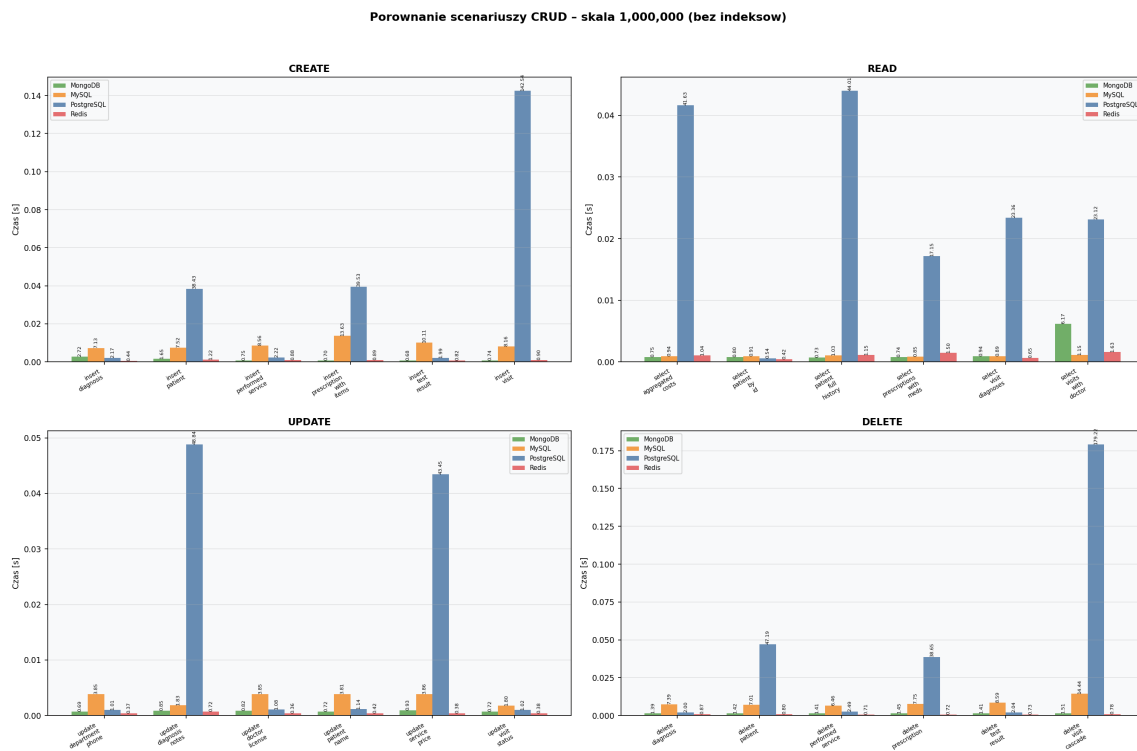
8 Wizualizacja wyników



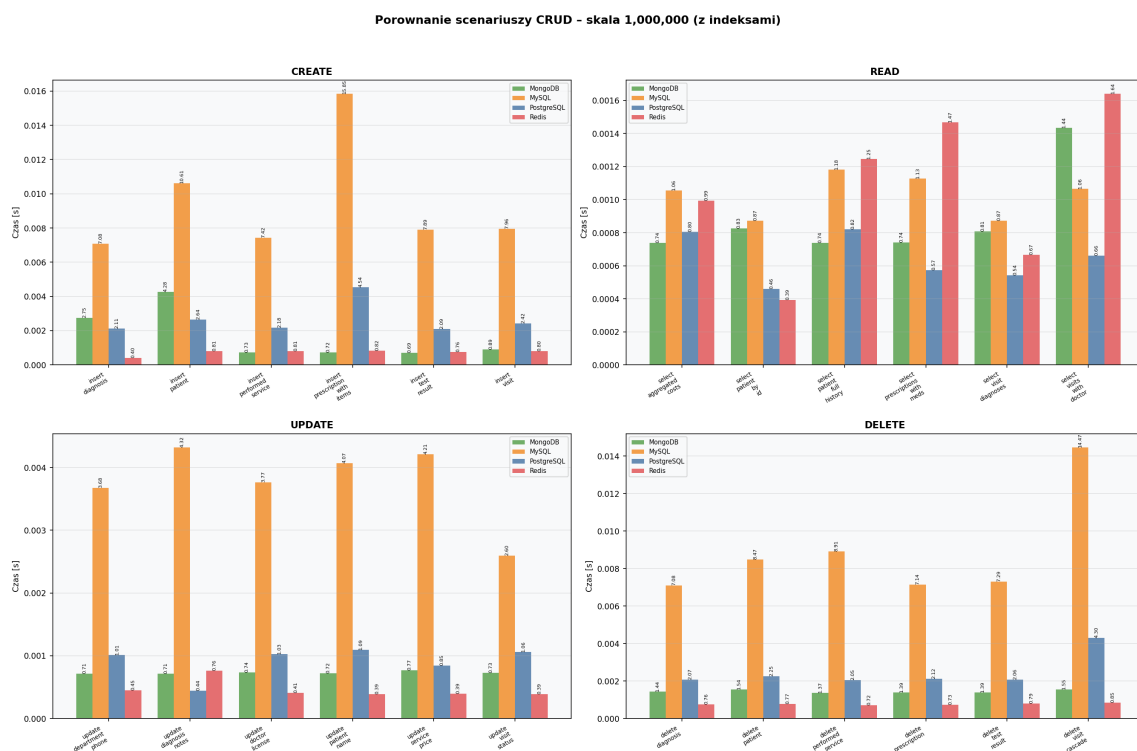
Rysunek 1: Porównanie scenariuszy CRUD – skala 10 000 (bez indeksów)



Rysunek 2: Porównanie scenariuszy CRUD – skala 100 000 (bez indeksów)

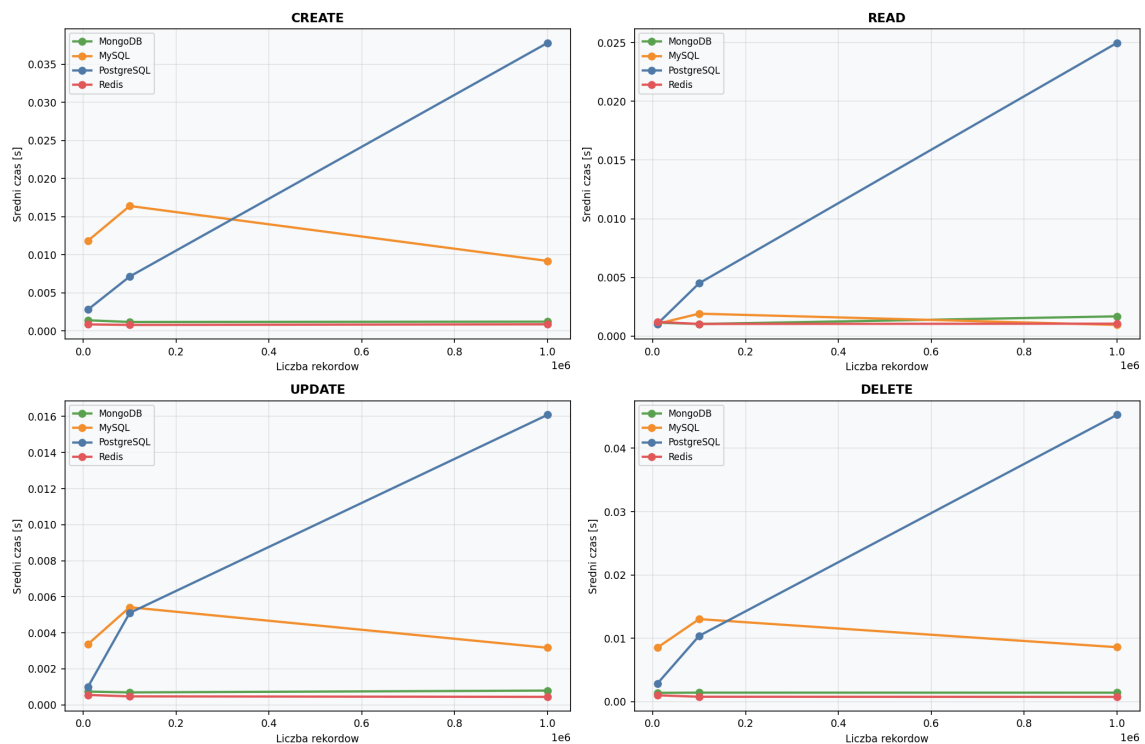


Rysunek 3: Porównanie scenariuszy CRUD – skala 1 000 000 (bez indeksów)



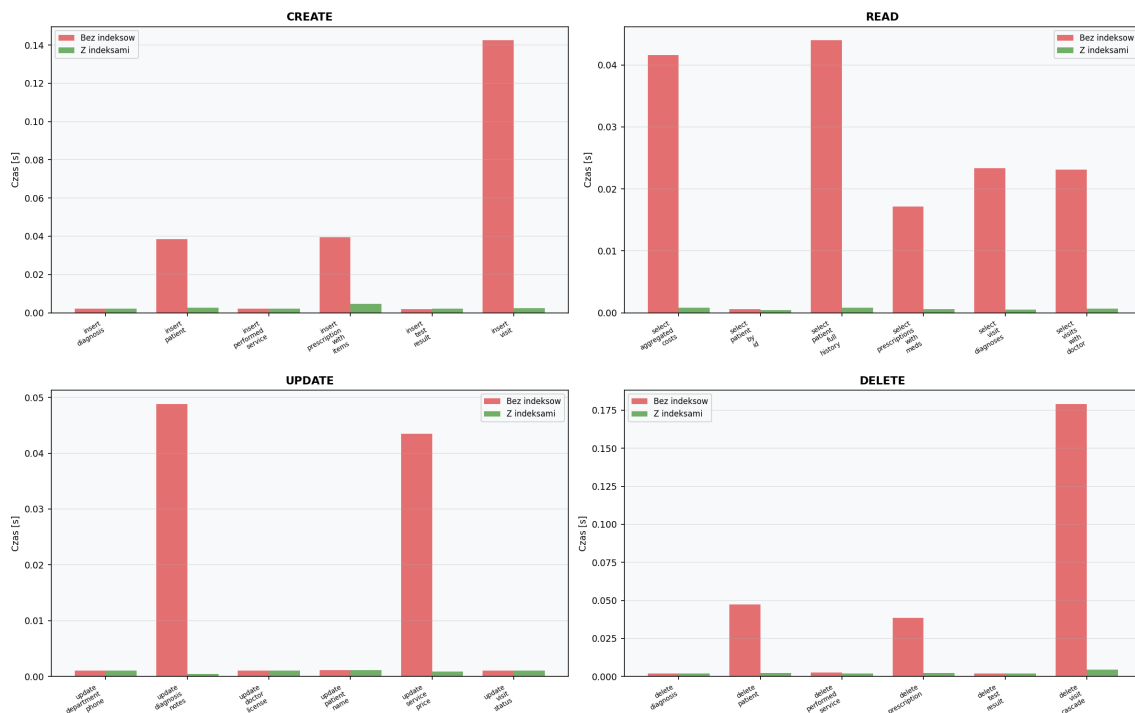
Rysunek 4: Porównanie scenariuszy CRUD – skala 1 000 000 (z indeksami)

Skalowalnosc - wpływ rozmiaru danych na wydajnosc (bez indeksow)

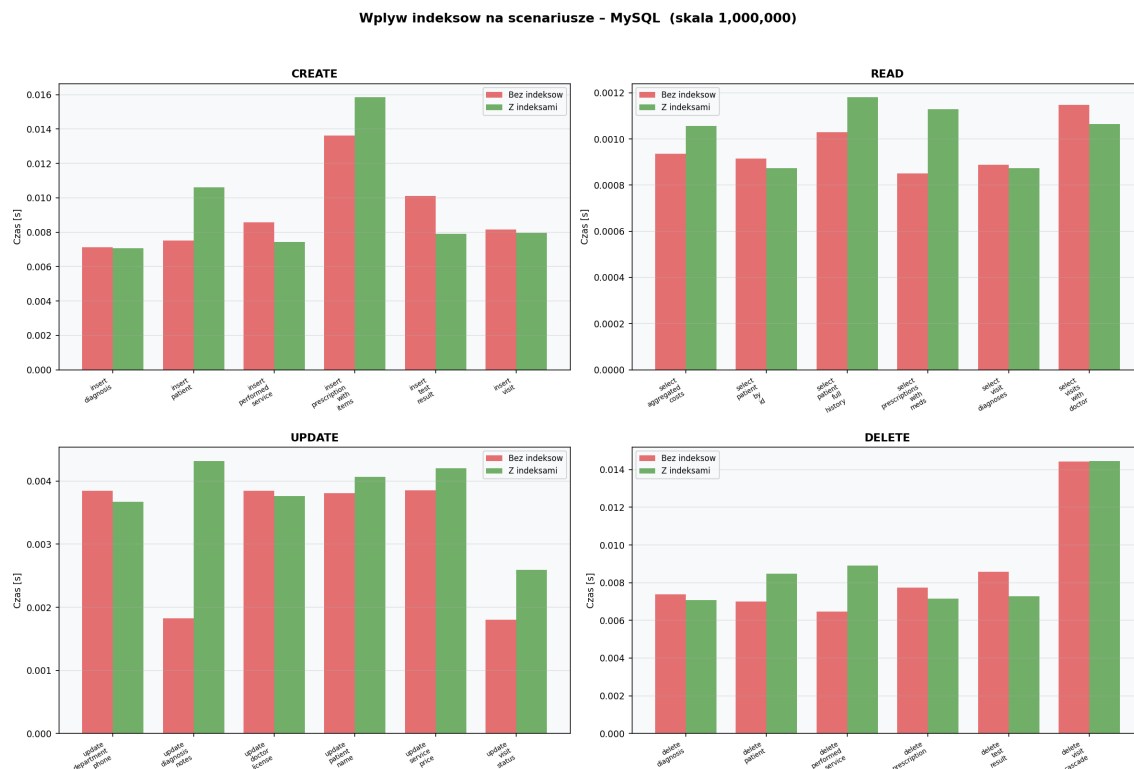


Rysunek 5: Skalowalność – wpływ rozmiaru danych na średni czas operacji (bez indeksów)

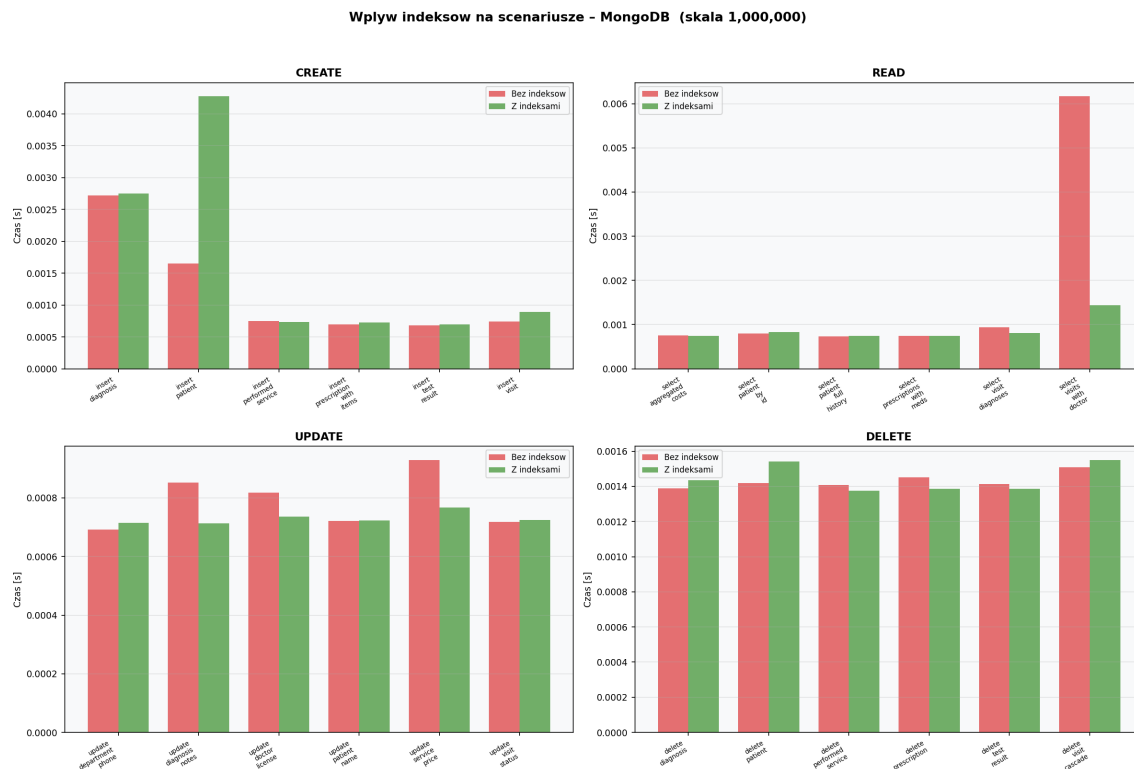
Wplyw indeksow na scenariusze - PostgreSQL (skala 1,000,000)



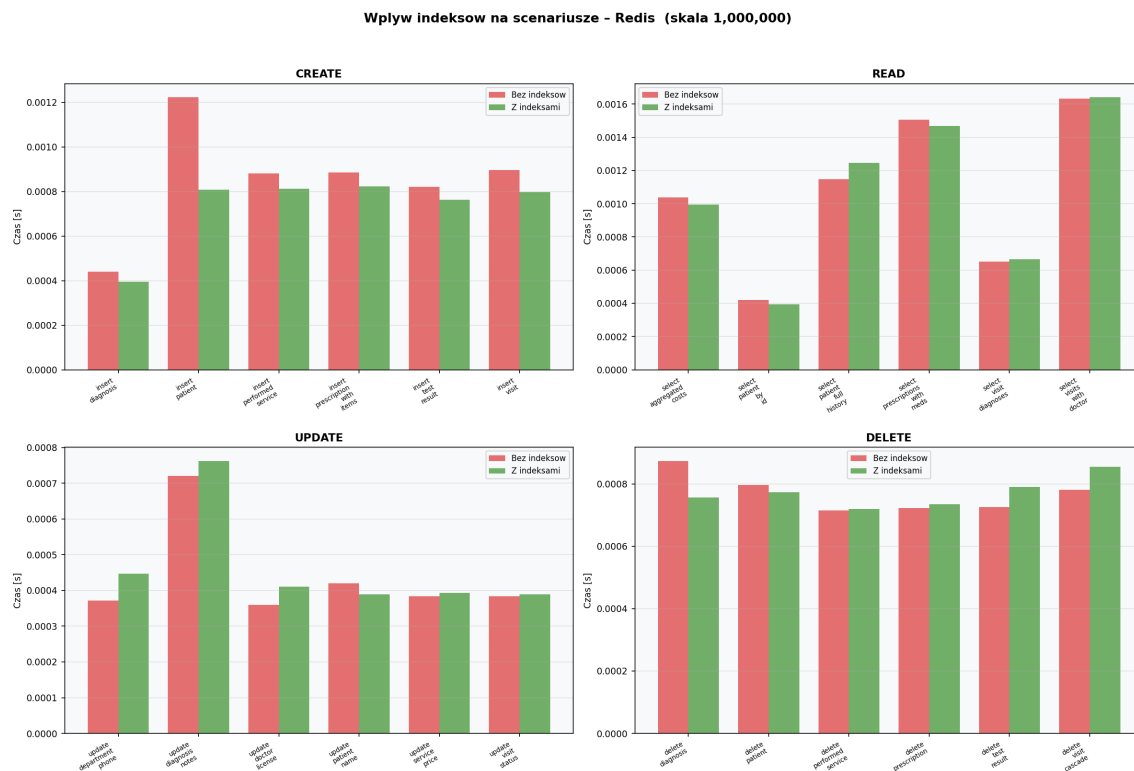
Rysunek 6: Wpływ indeksów na scenariusze – PostgreSQL (skala 1M)



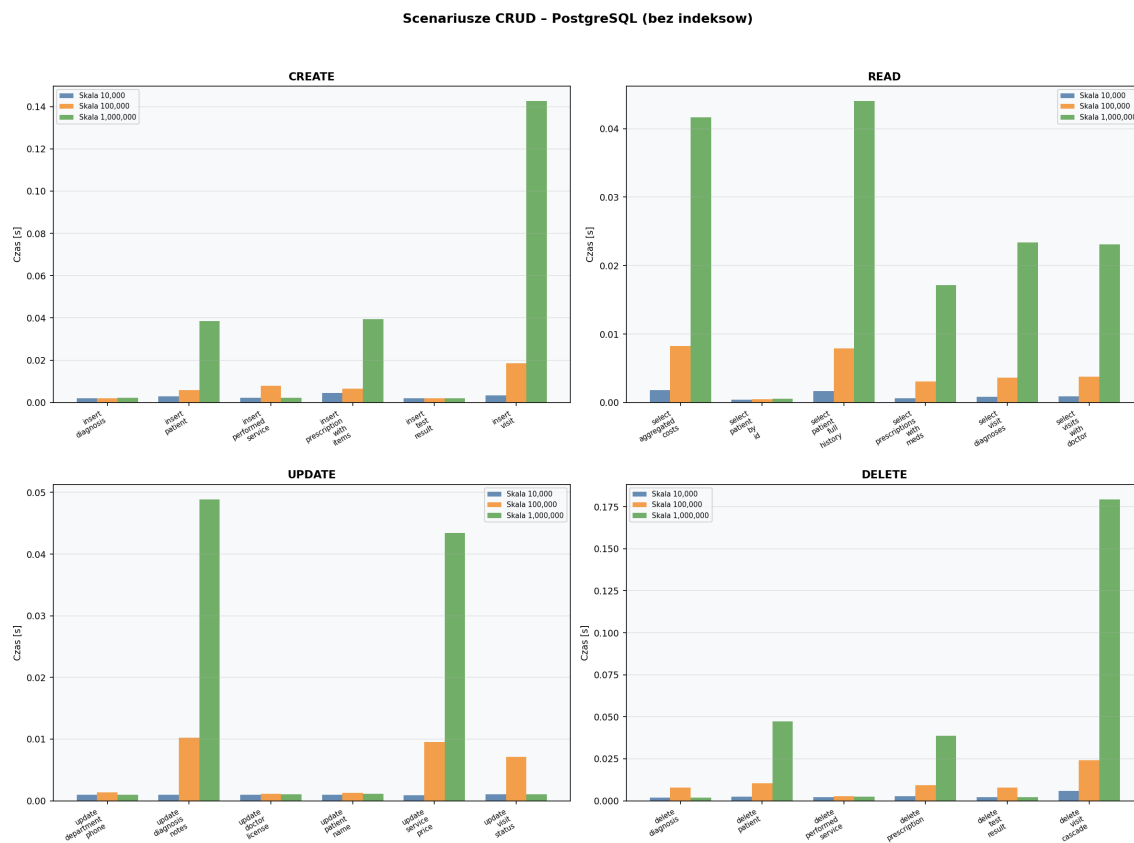
Rysunek 7: Wpływ indeksów na scenariusze – MySQL (skala 1M)



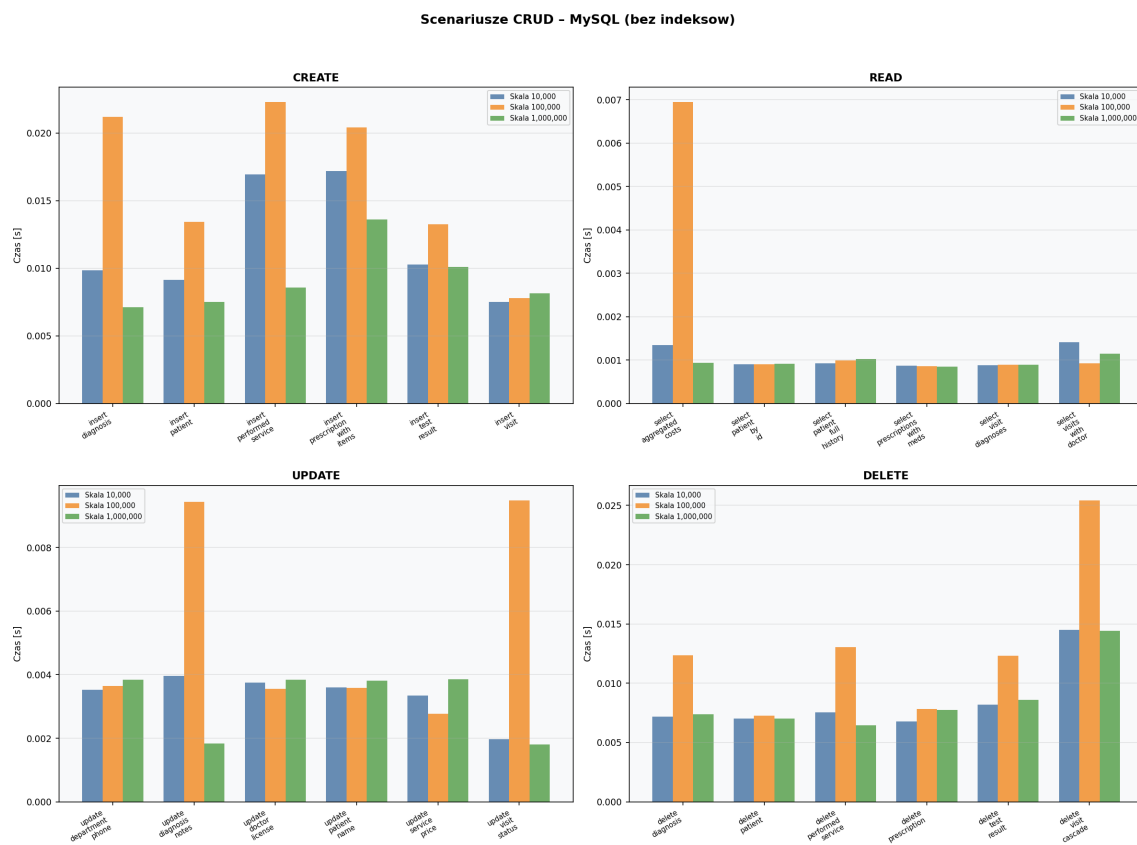
Rysunek 8: Wpływ indeksów na scenariusze – MongoDB (skala 1M)



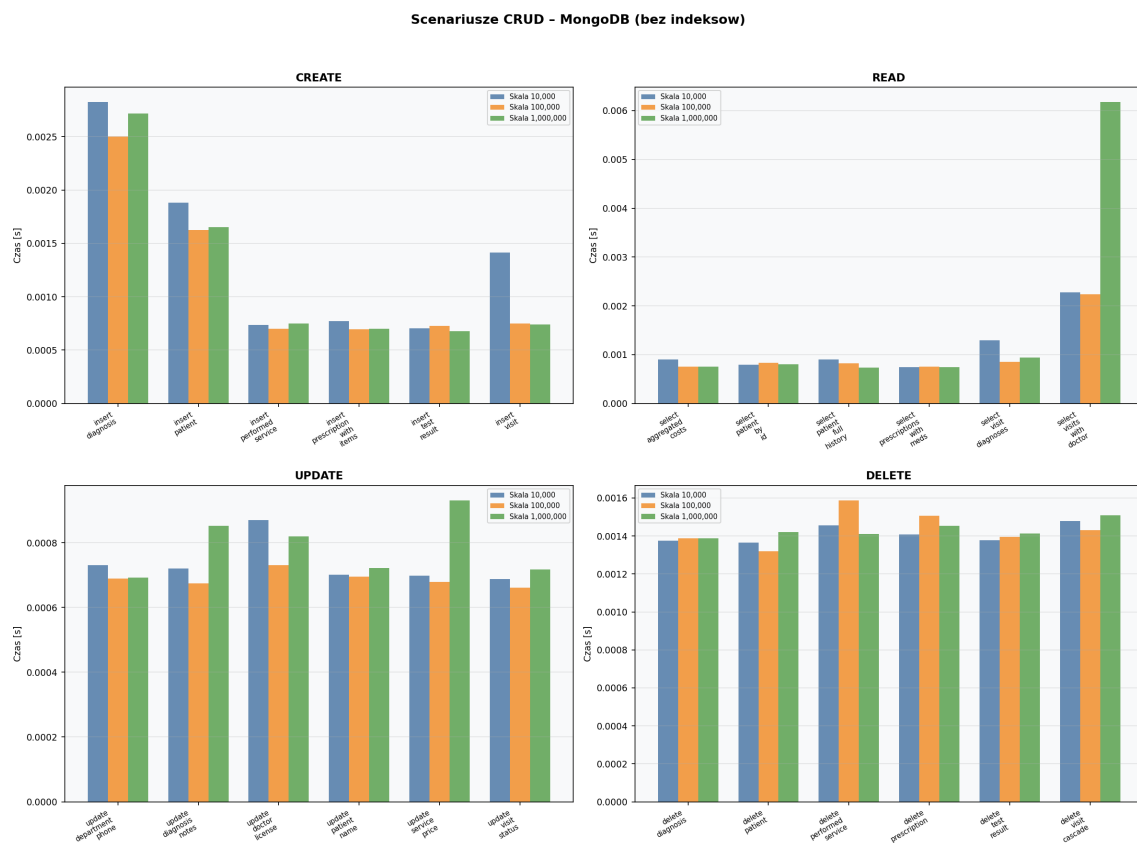
Rysunek 9: Wpływ indeksów na scenariusze – Redis (skala 1M)



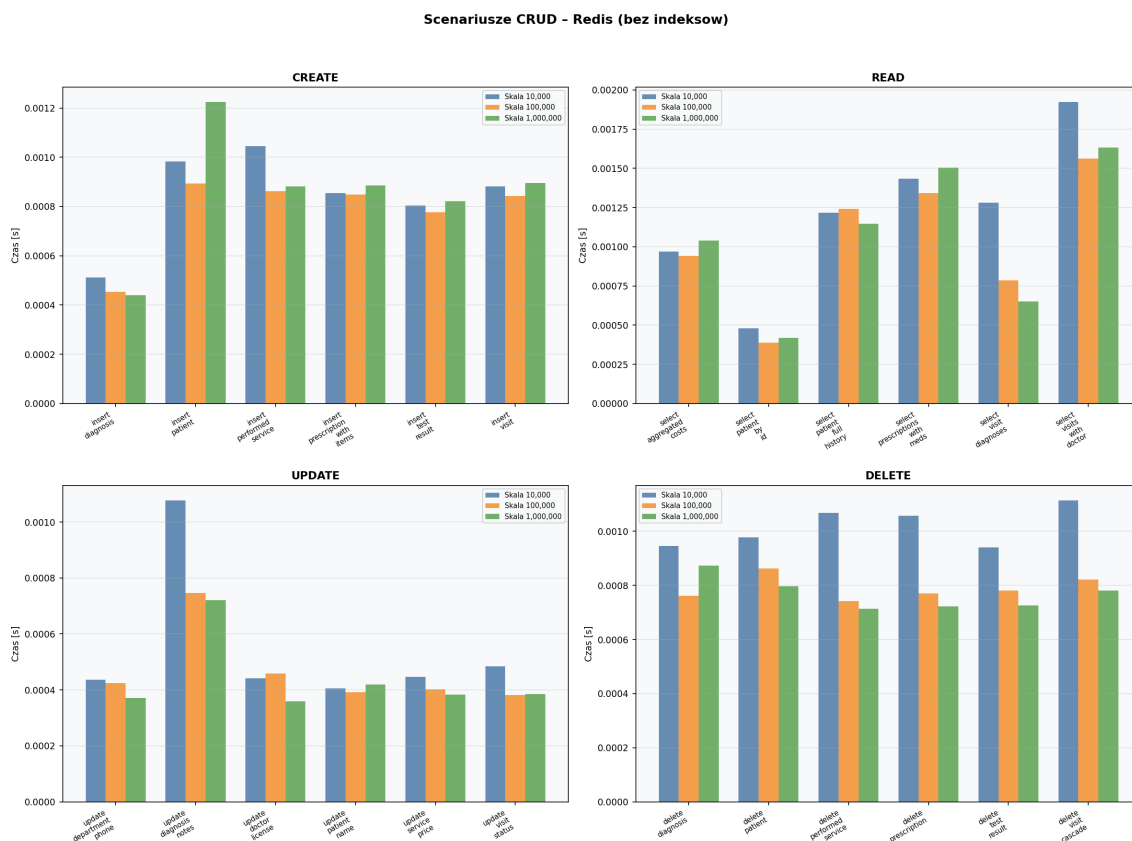
Rysunek 10: Scenariusze CRUD we wszystkich skalach – PostgreSQL



Rysunek 11: Scenariusze CRUD we wszystkich skalach – MySQL



Rysunek 12: Scenariusze CRUD we wszystkich skalach – MongoDB



Rysunek 13: Scenariusze CRUD we wszystkich skalach – Redis

9 Analiza indeksów

9.1 Zastosowane indeksy

W systemach relacyjnych utworzono 16 indeksów B-tree na kolumnach kluczy obcych i kolumnach filtrujących. W MongoDB utworzono 7 indeksów na kolekcji `patients` (w tym na polach zagnieżdżonych: `visits.status`, `visits.doctor_id`, `visits.diagnoses.disease_id`).

9.2 Analiza planów zapytań (EXPLAIN)

PostgreSQL – zapytanie R2 (JOIN wizyt z lekarzami, skala 1M, z indeksami):

```
Hash Join (cost=17.51..59.87 rows=11 width=33)
  Hash Cond: (v.doctor_id = d.id)
  -> Bitmap Heap Scan on visits v
        Recheck Cond: (patient_id = 1)
        -> Bitmap Index Scan on idx_visits_patient_id
  -> Hash -> Seq Scan on doctors d
Execution Time: 0.159 ms
```

Optymalizator wykorzystuje `Bitmap Index Scan` na `idx_visits_patient_id`, eliminując sekwencyjne skanowanie tabeli wizyt (1M wierszy).

MongoDB – `find po _id`: etap `IDHACK`, `totalDocsExamined`: 1, czas < 1 ms.

Redis – Redis nie posiada mechanizmu `EXPLAIN` ani planów zapytań. Dostęp do danych odbywa się bezpośrednio przez klucz ($O(1)$ `HGET/GET`) lub przez komendy

zbiorowe (SMEMBERS, LRANGE). Złożoność operacji jest deterministyczna i wynika z dokumentacji komend Redis, a nie z optymalizatora zapytań.

10 Weryfikacja hipotezy badawczej

H1: *Zastosowanie indeksów znacząco poprawia wydajność operacji SELECT kosztem spadku wydajności operacji INSERT i UPDATE.*

10.1 Analiza wyników – PostgreSQL (skala 1M)

Tabela 13: Weryfikacja hipotezy – PostgreSQL, skala 1 000 000

Scenariusz	Bez [ms]	Z idx [ms]	Zmiana	Wynik
insert_patient	38.43	2.64	−93%	przyspieszenie
insert_visit	142.54	2.42	−98%	przyspieszenie
select_visits_with_doctor	23.12	0.66	−97%	potwierdzenie H1
select_full_history	44.01	0.82	−98%	potwierdzenie H1
select_aggregated_costs	41.63	0.80	−98%	potwierdzenie H1
update_service_price	43.45	0.85	−98%	przyspieszenie
update_diagnosis_notes	48.84	0.44	−99%	przyspieszenie
delete_visit_cascade	179.22	4.30	−98%	przyspieszenie

10.2 Wnioski dotyczące hipotezy

Hipoteza H1 została **częściowo potwierdzona**:

1. **Potwierdzenie:** Operacje READ (SELECT) zyskały najwyższe przyspieszenie – do **99%** w PostgreSQL dla skali 1M. Indeksy eliminują sekwencyjne skanowanie na rzecz skanów indeksowych.
2. **Odrzucenie części hipotezy:** Wbrew oczekiwaniom, operacje INSERT i UPDATE w PostgreSQL **również przyspieszyły** po zastosowaniu indeksów. Wynika to z faktu, że:
 - Scenariusze INSERT (`insert_patient`, `insert_visit`) obejmują wstawianie z kluczami obcymi – indeksy na kolumnach referencyjnych przyspieszają walidację FK (zamiast Seq Scan: Index Scan).
 - Scenariusze UPDATE (`update_service_price`, `update_diagnosis_notes`) wyszukują wiersz po `visit_id` – z indeksem lokalizacja wiersza jest $O(\log n)$ zamiast $O(n)$.
3. **MySQL potwierdza klasyczny kompromis:** INSERT w MySQL jest wolniejszy z indeksami (`insert_patient`: +41%), co odpowiada kosztowi aktualizacji 16 struktur indeksowych przy każdym wstawieniu. MySQL nie wykazuje dramatycznego spadku READ bez indeksów, prawdopodobnie dzięki efektywnemu buforowaniu InnoDB.
4. **MongoDB:** Minimalny wpływ indeksów na większość operacji – model dokumentowy z dostępem po `_id` (IDHACK) jest naturalnie szybki. Wyjątek: `select_visits_with_doctor` przyspieszył o 77% dzięki indeksowi na `visits.doctor_id`.

5. **Redis:** Indeksy SQL/MongoDB nie mają wpływu na Redis – baza klucz-wartość operuje na adresowaniu bezpośrednim. Wahania w tabeli 12 ($\pm 9\%$) wynikają wyłącznie z naturalnego szumu pomiarowego. Redis konsekwentnie najszybszy we wszystkich kategoriach CRUD ($\sim 0.4\text{--}1.6$ ms), co potwierdza charakter bazy in-memory z operacjami $O(1)$.

11 Rozszerzona analiza wyników

11.1 Skalowalność

Na podstawie wyników i wykresu skalowalności (Rys. 5):

- **PostgreSQL bez indeksów:** Gwałtowny wzrost czasów READ i DELETE ze skalą (od 1.66 ms do 44.01 ms dla R4; od 5.94 ms do 179.22 ms dla D6). Z indeksami – prawie stały czas niezależnie od skali.
- **MySQL:** Stabilne czasy READ niezależnie od skali (bufor InnoDB), ale CREATE spowalnia nieregularnie.
- **MongoDB:** Stały czas dla większości operacji dzięki modelowi dokumentowemu. Wyjątek: `select_visits_with_doctor` rośnie z 2.28 ms do 6.17 ms.
- **Redis:** Niemal idealnie stałe czasy niezależnie od skali danych (od 10k do 1M). Operacje CRUD mieszczą się w przedziale 0.36–1.63 ms. Potwierdza to $O(1)$ złożoność operacji na tablicach haszujących w pamięci RAM.

11.2 Porównanie modeli danych

Model relacyjny (PostgreSQL, MySQL):

- Wymaga JOINów, ale z indeksami osiąga doskonałą wydajność (< 1 ms nawet przy 1M).
- Silne gwarancje spójności (ACID, klucze obce).
- PostgreSQL silnie zależny od indeksów – bez nich wydajność drastycznie spada ze skalą.

Model dokumentowy (MongoDB):

- Stała wydajność niezależnie od skali (odczyty po `_id`: ~ 0.8 ms).
- Eliminuje JOINy dzięki zagnieżdżeniom.
- Operacje `aggregate` z `$unwind` kosztowne na dużych dokumentach.

Model klucz-wartość (Redis):

- Najniższa latencja ze wszystkich testowanych baz – operacje pojedynczego klucza w ~ 0.4 ms.
- Brak JOINów wymaga wielokluczowych pipeline'ów (SMEMBERS $\rightarrow$ batch GET $\rightarrow$ batch HGETALL), co zwiększa latencję złożonych odczytów do ~ 1.5 ms.
- Brak indeksów – wydajność determinowana wyłącznie złożonością algorytmiczną komend.
- Dane przechowywane wyłącznie w RAM – ograniczona pojemność, ale brak opóźnień dyskowych.

11.3 Dane półustrukturalne (JSON)

MongoDB przechowuje dane w formacie BSON (Binary JSON), co stanowi wykorzystanie danych półustrukturalnych. Dokumenty pacjentów zawierają zagnieżdżone tablice wizyt, diagnoz, recept – bez sztywnego schematu.

11.4 Automatyzacja testów

Projekt realizuje pełną automatyzację: jeden skrypt `run_all.py` uruchamia kompletny pipeline (generowanie danych, wstawianie, benchmarki bez/z indeksami, EXPLAIN, wykresy).

12 Podsumowanie i wnioski

1. **Redis** osiąga najniższą latencję ze wszystkich testowanych baz – operacje CRUD w przedziale **0.36–1.63 ms** niezależnie od skali danych (10k–1M). Wynika to z operacji w pamięci RAM z $O(1)$ złożonością. Wadą jest brak relacji, brak JOINów i ograniczona pojemność.
2. **PostgreSQL z indeksami** oferuje najlepszą wydajność dla złożonych zapytań relacyjnych – operacje READ z wieloma JOINami w 0.5–0.8 ms nawet przy 1M wizytach. Bez indeksów PostgreSQL jest **najwolniejszy** ze wszystkich testowanych baz (do 179 ms).
3. **MongoDB** wykazuje stabilną, niską latencję (~ 0.7 –1.5 ms) niezależnie od skali i indeksów, dzięki modelowi dokumentowemu z dostępem po `_id`. Wadą jest wyższy koszt operacji na zagnieżdżonych tablicach.
4. **MySQL** zachowuje stabilne czasy dzięki buforowaniu InnoDB, ale jest wolniejszy od PostgreSQL z indeksami w operacjach z wieloma JOINami. CREATE konsekwentnie wolniejsze niż w pozostałych bazach (7–17 ms).
5. **Indeksy są kluczowe** dla PostgreSQL – eliminują Seq Scan, dając przyspieszenie **do 99%** w scenariuszach z JOINami i filtrowaniem po kolumnach niebędących PK. Co istotne, w PostgreSQL indeksy przyspieszają również INSERT (walidacja FK) i UPDATE (lokalizacja wiersza). Redis nie korzysta z indeksów – operacje bazują na bezpośrednim adresowaniu kluczy.
6. **Wybór SZBD** powinien być podyktowany charakterem aplikacji:
 - Złożone zapytania analityczne → PostgreSQL (z indeksami)
 - Proste aplikacje webowe → MySQL
 - Elastyczny schemat, niski czas odpowiedzi → MongoDB
 - Najniższa latencja, cache, sesje, kolejki → Redis